

```

# FCModel.py
import torch
import torch.nn as nn

class FCModel(nn.Module):
    def __init__(self, input_dim=768, hidden_dim=256, output_dim=1):
        """
        全连接分类模型
        Args:
            input_dim: BERT 输出维度 (768 for BERT-base)
            hidden_dim: 隐藏层维度
            output_dim: 输出维度 (1 for binary classification)
        """
        super(FCModel, self).__init__()

        self.classifier = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(hidden_dim // 2, output_dim),
            nn.Sigmoid() # 二分类使用 sigmoid 激活函数
        )

    def forward(self, x):
        return self.classifier(x).squeeze(-1) # 确保输出形状正确

```

```

# MRPCDataset.py
import torch
from torch.utils.data import Dataset
import pandas as pd

class MRPCDataset(Dataset):
    def __init__(self,
train_file="D:/vscode/BigDataAnalysisPractice/lab5/processed/mrpc_train.csv",

test_file="D:/vscode/BigDataAnalysisPractice/lab5/processed/mrpc_test.csv",
mode='train'):
        """
        初始化 MRPC 数据集
        Args:

```

```

        train_file: 训练集文件路径
        test_file: 测试集文件路径
        mode: 'train'或'test'
    """
    if mode == 'train':
        self.data = pd.read_csv(train_file)
    else:
        self.data = pd.read_csv(test_file)

    print(f"Loaded {mode} dataset with {len(self.data)} samples")

    # 显示数据列信息
    print("数据列:", self.data.columns.tolist())
    print("前 3 行数据:")
    print(self.data.head(3))

def __len__(self):
    return len(self.data)

def __getitem__(self, idx):
    row = self.data.iloc[idx]

    # 适应不同的列名
    if 'sentence1' in self.data.columns and 'sentence2' in self.data.columns:
        sentence1 = str(row['sentence1'])
        sentence2 = str(row['sentence2'])
    elif 'text1' in self.data.columns and 'text2' in self.data.columns:
        sentence1 = str(row['text1'])
        sentence2 = str(row['text2'])
    else:
        # 使用前两列作为句子
        sentence1 = str(row.iloc[0])
        sentence2 = str(row.iloc[1])

    # 标签列
    if 'label' in self.data.columns:
        label = int(row['label'])
    elif 'is_duplicate' in self.data.columns:
        label = int(row['is_duplicate'])
    else:
        label = int(row.iloc[2]) # 假设标签在第三列

    return sentence1, sentence2, torch.tensor(label, dtype=torch.float32)

```

```

# main.py
import torch
from torch.utils.data import DataLoader
from FCModel import FCModel
from MRPCDataset import MRPCDataset
from transformers import BertTokenizer, BertModel
import time
import os

def binary_accuracy(predict, label):
    """计算二分类准确率"""
    rounded_predict = torch.round(predict)
    correct = (rounded_predict == label).float()
    accuracy = correct.sum() / len(correct)
    return accuracy

def train_epoch(model, bert_model, train_loader, tokenizer, criterion, optimizer, bert_optimizer,
device):
    """训练一个 epoch"""
    model.train()
    bert_model.train()

    epoch_loss, epoch_acc = 0., 0.
    total_len = 0

    for i, (sentences1, sentences2, labels) in enumerate(train_loader):
        labels = labels.to(device)

        # 使用 BERT tokenizer 处理句子对
        encoding = tokenizer(
            list(sentences1),
            list(sentences2),
            return_tensors='pt',
            padding=True,
            truncation=True,
            max_length=128, # 限制长度以节省内存
            add_special_tokens=True
        )

        # 将数据移到设备
        input_ids = encoding['input_ids'].to(device)
        attention_mask = encoding['attention_mask'].to(device)

        # 前向传播

```

```

bert_output = bert_model(
    input_ids=input_ids,
    attention_mask=attention_mask
)

# 使用[CLS] token 的表示
cls_embedding = bert_output.last_hidden_state[:, 0, :]

# 分类
predictions = model(cls_embedding)

# 计算损失
loss = criterion(predictions, labels)
acc = binary_accuracy(predictions, labels)

# 反向传播
optimizer.zero_grad()
bert_optimizer.zero_grad()
loss.backward()

# 梯度裁剪
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
torch.nn.utils.clip_grad_norm_(bert_model.parameters(), max_norm=1.0)

optimizer.step()
bert_optimizer.step()

epoch_loss += loss.item() * len(labels)
epoch_acc += acc.item() * len(labels)
total_len += len(labels)

if i % 50 == 0:
    print(f"Batch {i}: Loss = {loss.item():.4f}, Accuracy = {acc.item():.4f}")

return epoch_loss / total_len, epoch_acc / total_len

def evaluate(model, bert_model, test_loader, tokenizer, criterion, device):
    """评估模型"""
    model.eval()
    bert_model.eval()

    test_loss, test_acc = 0., 0.
    total_len = 0

```

```

with torch.no_grad():
    for i, (sentences1, sentences2, labels) in enumerate(test_loader):
        labels = labels.to(device)

        encoding = tokenizer(
            list(sentences1),
            list(sentences2),
            return_tensors='pt',
            padding=True,
            truncation=True,
            max_length=128,
            add_special_tokens=True
        )

        input_ids = encoding['input_ids'].to(device)
        attention_mask = encoding['attention_mask'].to(device)

        bert_output = bert_model(
            input_ids=input_ids,
            attention_mask=attention_mask
        )

        cls_embedding = bert_output.last_hidden_state[:, 0, :]
        predictions = model(cls_embedding)

        loss = criterion(predictions, labels)
        acc = binary_accuracy(predictions, labels)

        test_loss += loss.item() * len(labels)
        test_acc += acc.item() * len(labels)
        total_len += len(labels)

        if i % 100 == 0:
            print(f"Eval Batch {i}: Loss = {loss.item():.4f}, Accuracy = {acc.item():.4f}")

    return test_loss / total_len, test_acc / total_len

```

```

def main():
    # 设置设备
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f"Using device: {device}")

    # 设置本地模型路径
    model_path = "D:/vscode/BigDataAnalysisPractice/lab5/bert-base-uncased"

```

```

try:
    # 载入数据集
    print("正在加载数据集...")
    train_dataset = MRPCDataset(mode='train')
    test_dataset = MRPCDataset(mode='test')

    train_loader = DataLoader(dataset=train_dataset, batch_size=16, shuffle=True)
    test_loader = DataLoader(dataset=test_dataset, batch_size=16, shuffle=False)
    print("数据载入完成")

    # 从本地路径加载 BERT 模型和 tokenizer
    print(f"正在从本地路径加载 BERT 模型: {model_path}")

    # 检查模型文件是否存在
    required_files = ['config.json', 'pytorch_model.bin', 'vocab.txt']
    for file in required_files:
        file_path = os.path.join(model_path, file)
        if os.path.exists(file_path):
            print(f"✅ 找到文件: {file}")
        else:
            print(f"❌ 缺少文件: {file}")

    # 加载 tokenizer
    tokenizer = BertTokenizer.from_pretrained(model_path)
    print(f"✅ Tokenizer 加载成功")

    # 加载 BERT 模型
    bert_model = BertModel.from_pretrained(model_path)
    bert_model.to(device)
    print(f"✅ BERT 模型加载成功")

    # 创建分类模型
    model = FCModel()
    model.to(device)
    print("分类模型创建完成")

    # 打印模型参数数量
    bert_params = sum(p.numel() for p in bert_model.parameters())
    fc_params = sum(p.numel() for p in model.parameters())
    print(f"BERT 模型参数: {bert_params:,}")
    print(f"分类模型参数: {fc_params:,}")
    print(f"总参数: {bert_params + fc_params:,}")

```

```

# 定义优化器和损失函数
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
bert_optimizer = torch.optim.Adam(bert_model.parameters(), lr=2e-5) # BERT 学习
率更小

criterion = torch.nn.BCELoss()

# 训练参数
num_epochs = 3
best_acc = 0.0

print("开始训练...")
for epoch in range(num_epochs):
    start_time = time.time()

    # 训练
    train_loss, train_acc = train_epoch(
        model, bert_model, train_loader, tokenizer,
        criterion, optimizer, bert_optimizer, device
    )

    # 评估
    test_loss, test_acc = evaluate(
        model, bert_model, test_loader, tokenizer, criterion, device
    )

    epoch_time = time.time() - start_time

    print(f"\nEpoch {epoch+1}/{num_epochs}:")
    print(f"  Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f}")
    print(f"  Test Loss: {test_loss:.4f}, Test Acc: {test_acc:.4f}")
    print(f"  Time: {epoch_time:.2f}s")

# 保存最佳模型
if test_acc > best_acc:
    best_acc = test_acc
    torch.save({
        'model_state_dict': model.state_dict(),
        'bert_model_state_dict': bert_model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'bert_optimizer_state_dict': bert_optimizer.state_dict(),
        'epoch': epoch,
        'best_acc': best_acc,
        'tokenizer_path': model_path
    }, 'best_bert_model.pth')

```

```

        print(f" ☒ 保存最佳模型，测试准确率: {best_acc:.4f}")

    print("-" * 60)

    print(f"\n 训练完成！最佳测试准确率: {best_acc:.4f}")

    # 最终模型测试
    print("\n=== 最终模型测试 ===")
    final_test_loss, final_test_acc = evaluate(
        model, bert_model, test_loader, tokenizer, criterion, device
    )
    print(f"最终测试结果 - Loss: {final_test_loss:.4f}, Accuracy: {final_test_acc:.4f}")

except Exception as e:
    print(f"发生错误: {e}")
    import traceback
    traceback.print_exc()

if __name__ == "__main__":
    main()

```

```

Epoch 1/3:
  Train Loss: 0.5640, Train Acc: 0.7174
  Test Loss: 0.5333, Test Acc: 0.7003
  Time: 862.46s
☒ 保存最佳模型，测试准确率: 0.7003

```

```

Epoch 2/3:
  Train Loss: 0.4484, Train Acc: 0.7949
  Test Loss: 0.4508, Test Acc: 0.7681
  Time: 865.37s
☒ 保存最佳模型，测试准确率: 0.7681

```

```

Epoch 3/3:
  Train Loss: 0.3710, Train Acc: 0.8457
  Test Loss: 0.4410, Test Acc: 0.8017
  Time: 869.14s
☒ 保存最佳模型，测试准确率: 0.8017

```



```
训练完成！最佳测试准确率：0.8017

=== 最终模型测试 ===
Eval Batch 0: Loss = 0.2789, Accuracy = 0.9375
Be aware, overflowing tokens are not returned for the setting you chose.
Be aware, overflowing tokens are not returned for the setting you chose.
Be aware, overflowing tokens are not returned for the setting you chose.
Be aware, overflowing tokens are not returned for the setting you chose.
Be aware, overflowing tokens are not returned for the setting you chose.
Eval Batch 100: Loss = 0.3078, Accuracy = 0.7500
最终测试结果 - Loss: 0.4410, Accuracy: 0.8017
```

Epoch-by-Epoch 进步：

Epoch	训练准确率	测试准确率	提升幅度
Epoch 1	71.74%	70.03%	基准
Epoch 2	79.49%	76.81%	+6.78%
Epoch 3	84.57%	80.17%	+3.36%